Esercitazione 3

Cataldo Basile
< cataldo.basile @ polito.it >

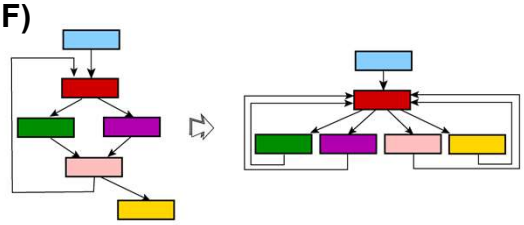
Politecnico di Torino
Dip. Automatica e Informatica

Software hacking experiment

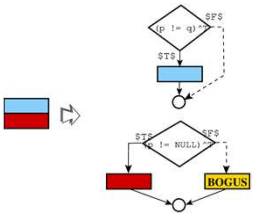
- **rispondere alla domanda: il software offuscato è realmente più complicato da capire e modificare rispetto a quello non offuscato?**
- **ognuno di voi ha provato a risolvere 3 problemi**
  - 1 applicazione vanilla (= non protetta)
  - 2 programmi offuscati
    - …con Tigress
    - http://tigress.cs.arizona.edu/index.html
- **per aiutarci a rispondere ad ulteriori domande**
  - ci sono maniere migliori di offuscare? Quanta parte della app mi conviene offuscare?
    - offuscare riduce sempre le performance
  - combinare più tipi di offuscamente è utile?

Software Obfuscation

- due tecniche usate per offuscare le app dell'esperimento…
- Control Flow Flattening (CFF)



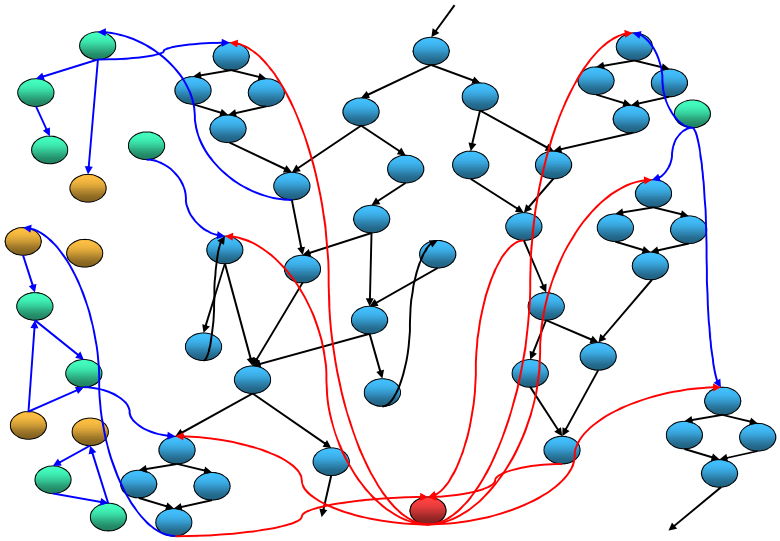
- Opaque Predicates



Misura con metriche oggettive

- misure inventate per la qualità e manutenzione del codice
  - linee di codice
  - complessità ciclomatica
    - quanti percorsi ci sono nel CFG tra inizio e fine del pezzo di codice in esame
  - misura di Halsted
    - conta quanti operatori vengono usati
- altre introdotte nella teoria anche per protezione SW
  - in pratica, se calcolano poche per mancanza di tool
  - https://pdfs.semanticscholar.org/9496/fd9801dd4b2ff9389308041d011ae1a77395.pdf
- assunzione implicità: più aumentano queste misure più è complicato da capire il programma
  - è vero? quanto?

Complessità ciclomatica



Metodi empirici

- **si misurano i tempi ed i successi mediante esperimenti che coinvolgono persone**
- **si correlano a diversi fattori / cofattori**
  - esperienza / abilità nella programmazione /
  - servirebbero miliardi di esperimenti con milioni di applicazioni / protezioni e livelli di protezione
    - per raggiungere un livello di copertura adeguato
- **limitato alle app ed alle protezioni usate**
  - non semplice generalizzare i risultati
    - e se cambia l'app?
    - e se cambio la protezione?
    - e se uso la stessa protezione con parametric diversi?
- **però qualcosa si può già vedere con un singolo esperimento**

Scopo dell'esperimento che vi abbiamo proposto

- **provare a correlare tempi e probabilità di successo a tipi di offuscamento con dati empirici**
  - …speriamo di trovare una misura della "obfuscation potency"
- **3 applicazioni diverse**
  - arithmetic, tictactoe, numbers
- **3 protezioni (+vanilla) = 4 casi**
  - CFF, OP, CFF+OP
- **12 gruppi diversi di campioni**
  - considerando l'ordine sono di più
- **cofattori da considerare**
  - anni di esperienza, C skill, metriche oggettive calcolate sulle applicazioni, ordine di ricevimento protezioni (learning)
- **analisi di processo: attack step**

ESEMPIO DI ANALISI DI SICUREZZA

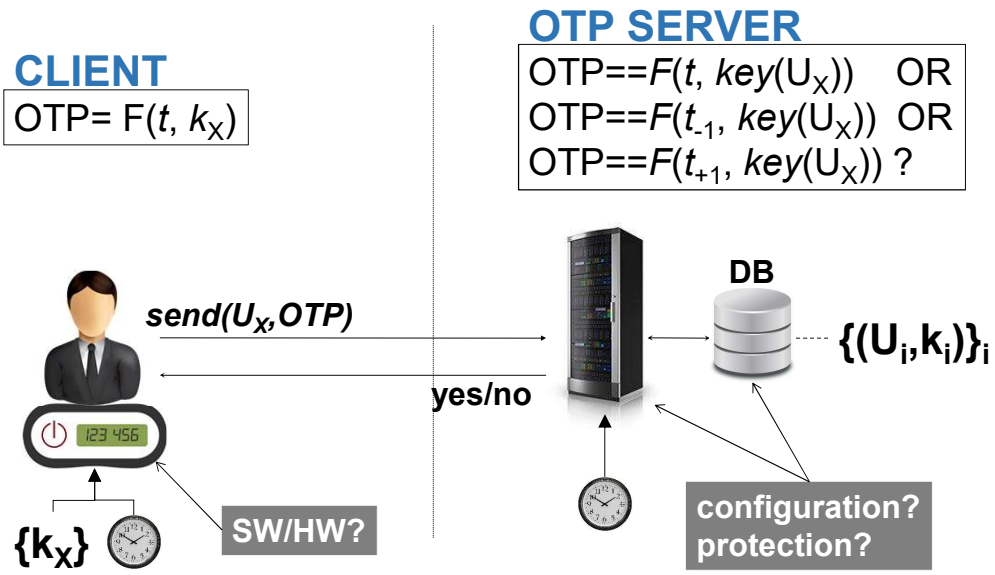
Analisi Time-based OTP

Un'azienda ha identificato TOTP come sistema ottimale per fornire autenticazione agli utenti.

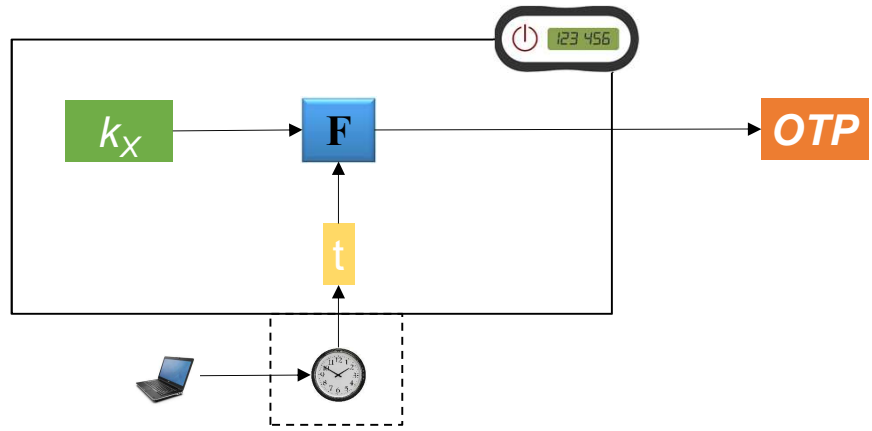
Svolgere un'analisi della soluzione considerando aspetti di

- **sicurezza**
- **usabilità**
- **implementabilità**
- **privacy**

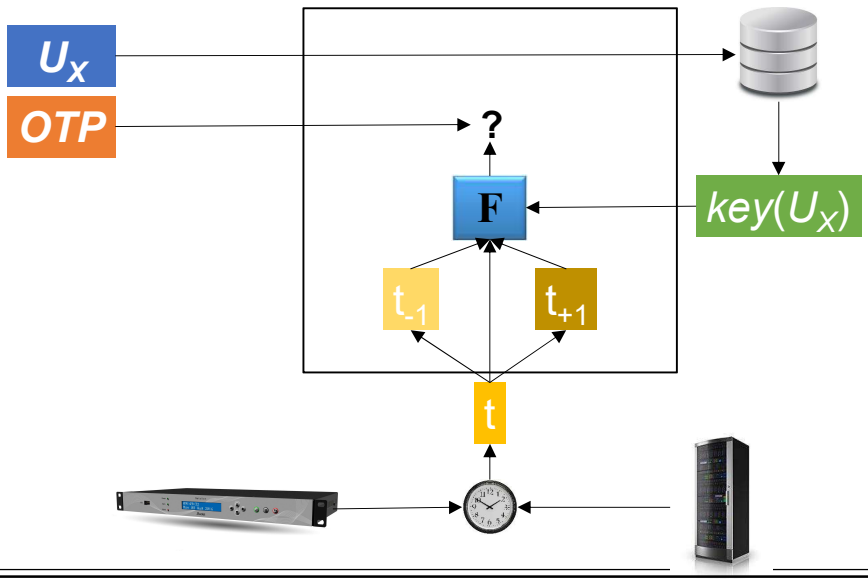
Time-based One Time Password



White-box model: the client



White-box model: the server



Risk analysis terminology (I)

- **assets**
  - things that have a value
- **threats**
  - deliberate actions or accidental events
  - exploit vulnerabilities
  - produce loss of security properties
- **vulnerabilities**
  - weaknesses of assets
  - make a system prone to attack by a threat or make an attack more likely to have success or impact

Risk analysis terminology (II)

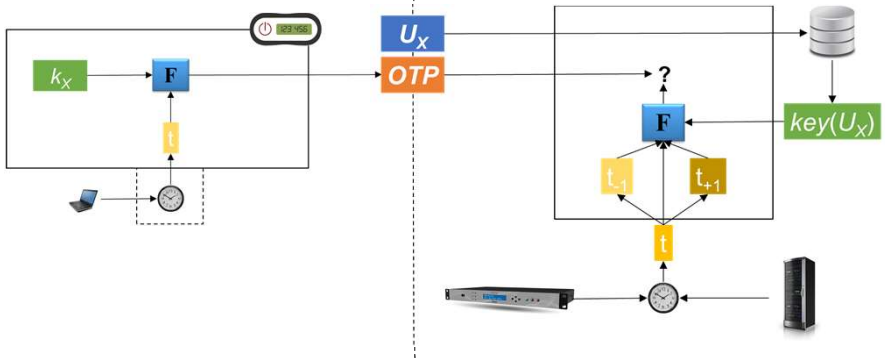
- **(security) controls**
  - countermeasures for vulnerabilities
  - deterrent
    - reduce the likelihood of deliberate attacks
  - preventive
    - protect vulnerabilities and reduce threat impact
  - corrective
    - reduce the effect of a threat / exploitation of vulnerabilities
  - detective
    - discover attacks (anomalies)
    - trigger preventive or corrective controls

Who are the enemies (threats)?

- **Man in the Middle attacks (MitM)**
  - passive: sniffing
  - active: spoofing
  - attack targets
    - the client
    - the server
- **Man at the End attacks (MatE)**
  - the end user alters the SW
- **human errors**
  - e.g., during normal use, management

Now take your time…

security
user-friendliness
implementation
privacy



Server-side (I)

- **server time synchronization**
  - threat: reply attack + time machine
    - 1) sniff the password at time *t* used by the legitimate user then 2) force the time *t* on the server
  - threat: DoS
    - change the time → no one can access
  - mitigations
    - ACL on the server for the users that can modify date and time
    - NTP but only with authentication
      - avoid fake NTP server to force wrong date and time

Server-side (II)

- **threat: alter the verification process**
  - tamper with the verification SW executed on the server
    - e.g., to accept invalid passwords
  - mitigation: protect the software
  - mitigation: protect the server from attacks that allow modifying the executed/stored software
- **threat: attacks that read the RAM**
  - dump the RAM → derive the user keys k_i
    - only when actually used, thus for patient attackers…
  - mitigation: use an HSM to store (*U,k*) pairs (more expensive)
  - mitigation (less effective): TrustZone o SGX to enforce strong process isolation

Server-side (III)

- **server hardening**
  - i.e., generic mitigations (independent from TOTP)
  - IDS
    - server needs to be accessed and bugs will eventually appear
  - firewall
    - block all the traffic but the authorized one
      - …have to allow at least the traffic to the server
  - antivirus, antimalware
    - other important security controls
  - patch management
  - use DMZ and consider NATting

Server-side (IV)

- **DB contains user/key pairs**
  - threats: attackers, accidental delete, DB corruption
  - vulnerabilities: to be found on the server, DB, OSes, etc.
    - look for CVE depending on the servers and the versions selected (design time)
  - sniff DB-Server communications (if DB is separated)
  - mitigations
    - strong access control to access the DB (not accessible from the outside if separate)
    - select software that has “good vulnerability records”
    - define and maintain a patch management system/procedure
    - ACL on the OS to limit privileges (CRUD)
    - backup against accidental or deliberate events that may destroy or render unusable the DB

In between: MITM attacks (threats)

- **threat: sniffing: NOT an issue**
  - e.g., replay
  - unless the attacker also controls the server time
- **threat: spoofing/filtering**
  - detectable with monitoring systems
  - DoS possible and unavoidable
- **threat: brute force attack**
  - depending on the OTP length and the time slot
  - 4 digits: attackers may try all of them in the 30s/60s time slot
  - usually 6 digits: 1 million combinations to try in the 30s/60s
    - risky (in crypto 2^{80} and 2^{128} are considered secure)
  - control: IDS monitors anomalies
    - e.g., tries too fast (people are not so fast), too many errors

In between: MITM attacks (threats)

- **threat: impersonation**
  - fake server answers instead of the legitimate one
    - forward OTPs to the correct server
      - and get access
  - mitigation: server authentication
    - e.g., TLS or a VPN
    - also helps in enforcing other security requirements

Client-side

- **TOPT token**
  - tamper proof HW
  - OK but expensive
- **threat: reverse engineering**
  - read the key if in the code or firmware
  - locate the functions that will manipulate the key
    - dynamical analysis / breakpoints
  - mitigation: make the code more complex to understand
    - yet maintain reasonable performance
  - mitigation: apply protections to avoid dynamic analysis
- **threat: change time**
  - force the client to produce OTP to be used in the future

Client-side

- **SW-based solutions (eg., app for smartphone/tablet/PC)**
  - exactly the same attack vectors as the server
  - read the key as soon as it is received from the OTP server (initialization)
    - change the OTP time *t* (easy as the user controls the PC)
    - extract *k* when it is in RAM (e.g., when it's actually used)
    - worse than on the server: clients are usually less protected than servers
    - reverse engineering
  - mitigations:
    - secure HW (e.g., ARM trustZone / Intel SGX)
    - on-line renewability of the app, frequent updates of the app
    - protect the code with software and data obfuscation + anti-tampering (+ diversification)

Usability / implementation

- **usability (user): reasonably good / easy to accept**
  - forget / loose the key
    - protect with the PIN (should be remembered)
- **implementation**
  - server-side: quite simple to implement
  - client-side/users: complicated yet feasible
    - select and buy tokens / OTP Software
    - define all the management procedures
  - SW-based OTP
    - app needs to be difficult to compromise
    - change it quite often

Privacy

- **TOTP does not pose issues on private, personal, sensitive data**
  - does not manage any of these data
  - check the local regulations
    - in EU → GDPR
- **risks if the same authentication server is used for access to more services**
  - profile users' behaviours from information about authentication requests